

Contributing

Contents

- [Style Guide](#)
- [Source Code](#)
- [Documentation](#)

This project is currently privately maintained in Sandia's GitLab server at [cee-gitlab.sandia.gov/joshart/sabl](https://gitlab.sandia.gov/joshart/sabl). Please [contact Joey Hart](#) if you are interested in contributing.

Style Guide

Code Formatting

MATLAB code in this repository is formatted using the `mh_style` command line tool provided in the [MISS_HIT](#) package by Florian Schanda.

```
python3 -m pip install miss_hit
```

Use the following commands to format the MATLAB source code.

```
mh_style          # List style problems in detail without changing any files.
mh_style --brief  # List style problems briefly without changing any files.
mh_style --fix    # Fix style problems, overwriting offending files.
```

The following will format MATLAB code blocks in the documentation.

[Back to top](#)

```
cd docs
python3 format_code_blocks.py
```

To ensure proper formatting before committing changes, install a git `pre-commit` hook with the following command.

[Skip to main content](#)

```
cp .pre-commit-formatting-hook .git/hooks/pre-commit
```

Now `git commit` automatically runs the formatter and cleans the code. A commit is **aborted** if the formatter makes any changes; in this case, review / add the changes and commit again.

To run the formatter without a commit:

```
./pre-commit-formatting-hook
```

Coding Conventions

- Class and function names: `Capitalize_And_Underscore`. The use of “math names” is approved so long as the notation is sufficiently documented. For example, the `Dynamic_Constraint` class has a `Time_Instance_RHS()` that is being renamed to simply `f()`.
- Hessian-vector product methods often have many arguments, representing the state $\mathbf{u}$, the control $\mathbf{z}$, sometimes the adjoint state $\boldsymbol{\lambda}$, and the search direction $\mathbf{v}$. Our convention is to list the search direction first, then the (state, control, adjoint) triple (or (state, control) pair if the adjoint is not involved). The state should always be `u`, the control `z` (or `y` and `q` in the dynamic setting, when appropriate), and the adjoint `lambda`. The search direction $\mathbf{v}$ should be `u_in` or `z_in`, depending on the size of $\mathbf{v}$; likewise, the return variable should be `u_out` or `z_out`.
- Classes, functions, and methods are documented using python’s `sphinx` framework with the `sphinxcontrib.matlab` extension. See [below](#) for more details.

TODO

- Naming conventions for tests: folders, drivers, etc.
- Error messages in tests
- Directory structure for examples?

Source Code

SABL codes are split into three main directories:

- `src/`, the problem-agnostic source code to be distributed as a package,
- `tests/`, basic unit and regression tests for the `src/` code, and
- `examples/`, a collection of fully fleshed-out applications.

[Skip to main content](#)

TODO

Mermaid diagram of the different modules.

Documentation

The source files for this documentation, and the scripts used to compile them into HTML format, are located in the `docs/` folder of the source repository. The `docs/` folder contains the following.

-  `Makefile`: Makefile for the documentation. Run `make help` for options and `make docs` to compile the documentation.
- `source/`: source files for the documentation.
- `build/`: directory containing compiled HTML files. This folder is not tracked by git.
-  `requirements.in`: high-level Python requirements for compiling the web-based documentation.
-  `requirements.txt`: exhaustive Python requirements for compiling the web-based documentation.

Building the Documentation

The documentation is built with the `sphinx` Python infrastructure, see [the sphinx documentation](#).

```
python3 -m pip install -r requirements.txt
make docs
```

To see the generated documentation, open `build/html/index.html` in a browser.

! Important

WOLF defines many abstract classes with methods that must be implemented by the user. Unfortunately, `sphinxcontrib_matlabdomain` does not currently support documenting abstract methods. The command `make docs` wraps the usual sphinx compiler by running a script that

1. switches to a new git branch
2. makes a few changes to the source code
3. runs the sphinx documentation builder, and
4. reverts to the previous git state.

This allows us to automatically generate documentation for abstract methods from their docstrings. **However**, this method ignores any current changes that are not committed to the git history. There are therefore two modes for generating documentation:

- `make docs`: generate documentation, including for abstract methods, from the last git commit.
- `make html`: generate documentation, **excluding** abstract methods, from the current files.

In short, use `make html` for testing small changes to the documentation and `make docs` for official deployments.

Writing Documentation

Documentation pages can be written in [reStructuredText](#) (`.rst`) or (preferred) in [Markdown](#) (`.md`).

The file [docs/source/conf.py](#) handles most of the sphinx configuration.

i Dependencies

If more Python packages (e.g., sphinx extensions) are needed,

- add them (with version numbers) to [docs/requirements.in](#)
- install `pip-compile`: `python -m pip install pip-tools`
- run `make requirements` to update [docs/requirements.txt](#).

! Important

The most up-to-date `sphinx==7.2.6` conflicts with `sphinx-book-theme==1.0.1`, so we currently use `sphinx==6.2.1`.

Source Files

Documentation source files are contained in the **docs/source/** directory, which has the following structure.

```
source/
├── conf.py                # Sphinx configuration file.
├── index.md              # Root of documentation (home page).
├── _static/              # CSS/HTML configuration.
├── problems/             # Descriptions of outer-loop problems we treat.
│   ├── bayesinversion.md
│   ├── hdsa.md
│   ├── oed.md
│   └── optimization.md
├── sabl/                 # Documentation for SABL (MATLAB).
│   ├── about.md          # SABL landing page.
│   ├── installation.md    # How to install SABL.
│   ├── guides.md          # Table of contents for code structure guide.
│   └── guides/            # Folder of code structure guides.
│       ├── bayesinversion.md
│       ├── hdsa.md
│       ├── oed.md
│       └── optimization.md
├── tutorials.md          # Table of contents for examples.
├── tutorials/            # Folder of examples.
│   ├── example1.md
│   ├── example2.md
│   └── ...
├── api.md                # Table of contents for API.
├── api/                  # Folder of API snippets.
│   └── ...
├── mrhyde/               # Documentation for MrHyDE (C++).
│   ├── about.md          # MrHyDE landing page.
│   ├── installation.md    # How to install MrHyDE.
│   ├── primer.md          # Transitioning from MATLAB to C++.
│   ├── guides.md          # Table of contents for code structure guide.
│   └── guides/            # Folder of code structure guides.
│       ├── bayesinversion.md
│       ├── hdsa.md
│       ├── oed.md
│       └── optimization.md
├── tutorials.md          # Table of contents for examples.
├── tutorials/            # Folder of examples.
│   ├── example1.md
│   ├── example2.md
│   └── ...
```

[Skip to main content](#)

Automatic API Generation

The documentation uses `sphinxcontrib_matlabdomain` to automatically generate the public API. This is nice, but the package has the following limitations:

- Abstract methods are not documented (use `make docs` to use our current workaround).
- Cannot generate the usual links to source code via `sphinx.ext.viewcode` that you often see with Python projects.
- Docstrings must be written in a Python-like format (we use [NumPy style](#)).
- The package only recognizes `"obj"` and `"self"` as the self-referential argument for class methods, but we are using `"this"` to prime users for C++. This can be fixed with a tiny hack:
 1. After installing `sphinxcontrib_matlabdomain`, find the file `mat_documenters.py`, which is probably somewhere like `<python environment>/lib/python3.<version>/site-packages/sphinxcontrib/mat_documenters.py`.
 2. Change the statement in line `1344` to the following.

```
if self.object.args[0] in ("obj", "self", "this") and not is _ctor:
```

To generate nice documentation from in-code docstrings, use the following syntax.

```
function [out1, out2] = Function_Name(this, arg1, arg2, arg3)
% Short description of the function. Write LaTeX with :math:`e = mc^2`.
%
% Display-style math blocks can be included as well;
% be careful to use this exact amount of indentation!
%
% .. math::
%   e = mc^2
%   \quad
%   A = \pi r^2
%
% Use :class:`My_Class` to refer to a class, :meth:`My_Class.My_Method`
% to refer to a method, and double backticks to write ``code``.
%
% Parameters
% -----
% arg1
%   Description of arg1, e.g., State :math:`\u \in \R^{n_u}`.
% arg2
%   Description of arg2, e.g., Control :math:`\v \in \R^{n_v}`.
```

[Skip to main content](#)

```
%
% Returns
% -----
% out1 : return_type
%   Description of the first return value.
% out2 : return_type
%   Description of the second return value.
out1 = 0;   % Actual code here.
out2 = 1;
```

Code comments *above* the function definition will not be parsed; only functions with this kind of docstring generate API documentation. The only exception is abstract class methods, which can be documented as follows.

```
classdef < My_Class

    methods (Abstract)

        out = My_Abstract_Method()
        % The docstring for an abstract method
        % matches the indentation of the function definition
        % because there is no ``end`` to delimit the body.

    end

end
```

Useful MyST Links

These are helpful for writing the documentation in Markdown.

- [MyST Markdown Guide](#)
- [Admonitions and Tabs](#) and [Admonitions in the Book Theme](#)
- [Tables](#)
- [Math and Equations](#)
- [Cross References](#)
- [Theorems, Proofs, and Algorithms](#)

In `conf.py` we also define a few LaTeX shortcuts that can be used anywhere in the documentation (including in the automatically generated API).

< [Previous](#)
[Notation](#)